

DisrtuCare — Communication Logiciel ↔ Matériel

Comment l'application mobile et le distributeur dialoguent

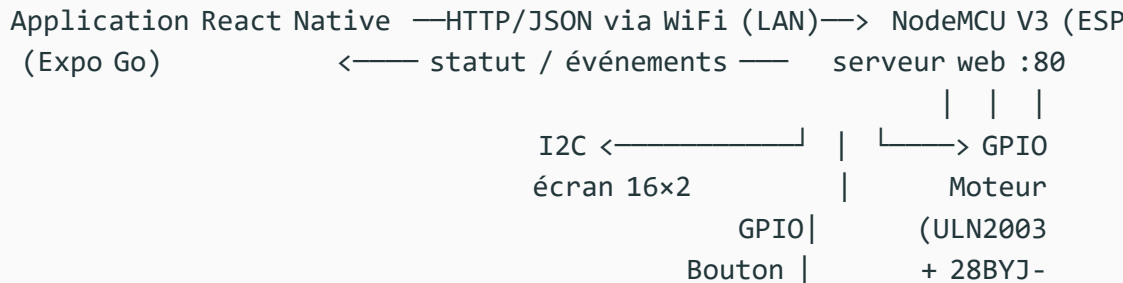
DisrtuCare — Note technique

Juin 2026

Vue d'ensemble

En une phrase

L'application et le distributeur de pilules sont **deux pairs HTTP sur le même réseau WiFi**. Le distributeur est un petit serveur web; l'application est son unique client.



48)

- **Pas de Bluetooth, pas de module natif, pas de cloud.** Juste `fetch()` sur le

Pourquoi HTTP via WiFi

Les choix de conception qui définissent le protocole

Décision	Conséquence
HTTP/JSON sur le port 80	Débogable depuis n'importe quel navigateur; aucun SDK sur le téléphone
Appareil = serveur, app = client	L'app trouve l'appareil par son IP , pas l'inverse
REST sans état + interrogation	Aucun socket persistant à maintenir ou reconnecter
CORS activé (*)	Fonctionne depuis le web/Expo sans souci d'en-têtes
Horaire stocké en EEPROM	L'appareil survit aux redémarrages sans l'app

*L'application n'a jamais besoin d'être connectée — elle **configure** et **observe**.
Toute la logique critique (distribution, confirmation) réside dans l'appareil.*

Les points d'accès de l'API

Sept endpoints sur le NodeMCU

Endpoint	Méthode	Rôle
/status	GET	Interrogation : horaire, heure et dernier événement
/sync	POST	Envoie les heures AM/PM + le nom du médicament (→ EEPROM + écran)
/settime	POST	Règle l'horloge de l'appareil
/led	POST	Allume / éteint la LED indicatrice
/motor	POST	Fait tourner le distributeur de N crans (manuel/test)

Cycle de connexion

Ce qui se passe quand on appuie sur « Connecter »

`httpService.connect()` exécute une poignée de main fixe :

1. **Résoudre l'URL** — `setUrl("192.168.1.42")` → `http://192.168.1.42`
2. **Ping** GET `/status` avec un délai de 6 s. Pas de réponse = « Déconnecté ».
3. **Synchroniser l'horloge** — POST `/settime` avec la date et l'heure courantes du téléphone (cette version n'a pas de RTC sauvegardée par pile).
4. **Démarrer l'interrogation** — GET `/status` toutes les **3 secondes**.
5. Émettre `connected`; l'IP est enregistrée en SQLite pour le prochain lancement.

Les étapes 2–3 garantissent : dès que l'app se connecte, l'horloge de l'appareil est juste et ses distributions programmées se déclenchent à la bonne heure locale.

Interrogation & modèle d'événements

Comment l'appareil « pousse » sans push

Il n'y a pas de socket — l'appareil tient un **compteur monotone**.

GET /status →

```
{  
  "am": "08:00", "pm": "20:00", "med": "Metformin",  
  "time": "14:30", "rtc": true, "awaiting": false,  
  "event_id": 3,  
  "last_event": { "evt": "confirmed", "type": "AM" }  
}
```

- Chaque événement matériel incrémente **event_id** et stocke last_event.
- L'app retient le dernier id vu. Quand event_id **change**, elle déclenche le gestionnaire de last_event une seule fois — ni doublon, ni perte.
- Une interrogation échouée bascule automatiquement l'app en **Déconnecté**.

Types d'événements

Trois choses que l'appareil peut signaler

evt	type	Déclenché par
dispensed	AM / PM	Heure programmée atteinte → moteur tourné
dispensed	TEST	POST /motor manuel depuis l'app
confirmed	AM / PM / TS	L'utilisateur a appuyé sur le bouton physique
error	—	Panne matérielle (le champ msg décrit)

- dispensed → l'app affiche « pilule libérée, en attente de confirmation ».
- confirmed → le tableau de bord **enregistre la dose comme Prise** et alerte l'utilisateur

Écrire vers l'appareil

Les corps des requêtes POST

Synchroniser l'horaire — POST /sync

```
{ "am": "08:00", "pm": "20:00", "med": "Metformin" }
```

Décomposé en amHour/amMin/pmHour/pmMin, écrit en EEPROM, affiché sur l'écran.

Régler l'horloge — POST /settime

```
{ "h": 14, "m": 30, "s": 0, "day": 16, "month": 6, "year": 2026 }
```

Actionneurs manuels — POST /motor et POST /led

```
{ "slots": 1 }           { "state": 1 }
```

Chaque POST répond 200 {"ok":true}, ou 400 {"ok":false,"error":"bad JSON"}.

De bout en bout : une dose programmée

Du tic d'horloge au « Prise » dans l'app

```
[appareil] loop(): heure rtc == heure AM → dispense("AM")  
    └─ motorRotate(1)  (2048 demi-pas ≈ ½ tour, un cran)  
    └─ écran : « Distribution » → « Terminé »  
    └─ la LED se met à clignoter, awaiting = true  
    └─ pushEvent {evt:"dispensed", type:"AM"} (event_id++)
```

```
[app]      la prochaine interrogation /status voit le nouvel event_id  
    └─ affiche « Dose libérée – veuillez confirmer »
```

```
[appareil] appui sur le bouton → awaiting = false, LED fixe 1,5 s  
    └─ pushEvent {evt:"confirmed", type:"AM"} (event_id++)
```

```
[app]      prochaine interrogation → enregistre la dose PRISE + alerte
```

Robustesse intégrée au protocole

Les détails qui le rendent fiable

- **Délais partout** — `AbortSignal` de 6 s sur chaque requête; un appareil bloqué ne fige jamais l'interface.
- **Itinérance multi-WiFi** — l'appareil connaît plusieurs réseaux (`ESP8266WiFiMulti`) et rejoint le meilleur disponible; revérifie toutes les ~8 s en cas de coupure.
- **Persistance EEPROM** — l'horaire et le nom du médicament survivent à une coupure (l'octet magique `0xAB` valide le bloc stocké).
- **Distribution idempotente** — le garde `lastDispensedMin` empêche une dose de se déclencher deux fois dans la même minute; réinitialisé à minuit.
- **Préliminaire CORS** — `OPTIONS` géré pour chaque route `POST`.
- **Récupération manuelle** — `/diag` et `/selftest` permettent à l'app de sonder le matériel sans cycle de distribution complet.

Synthèse

Le modèle de communication en cinq points

1. **Transport** : simple HTTP + JSON sur le WiFi partagé — rien d'autre.
2. **Rôles** : le distributeur est le serveur (:80), l'app est l'unique client.
3. **App** → **appareil** : écritures REST (/sync, /settime, /led, /motor).
4. **Appareil** → **app** : un /status interrogé avec un event_id croissant qui livre les événements dispensed / confirmed / error.
5. **Autonomie** : l'appareil distribue et confirme seul; l'app ne fait que le configurer et refléter ce qui s'est passé.

Le téléphone configure. L'appareil agit. Le statut réconcilie.